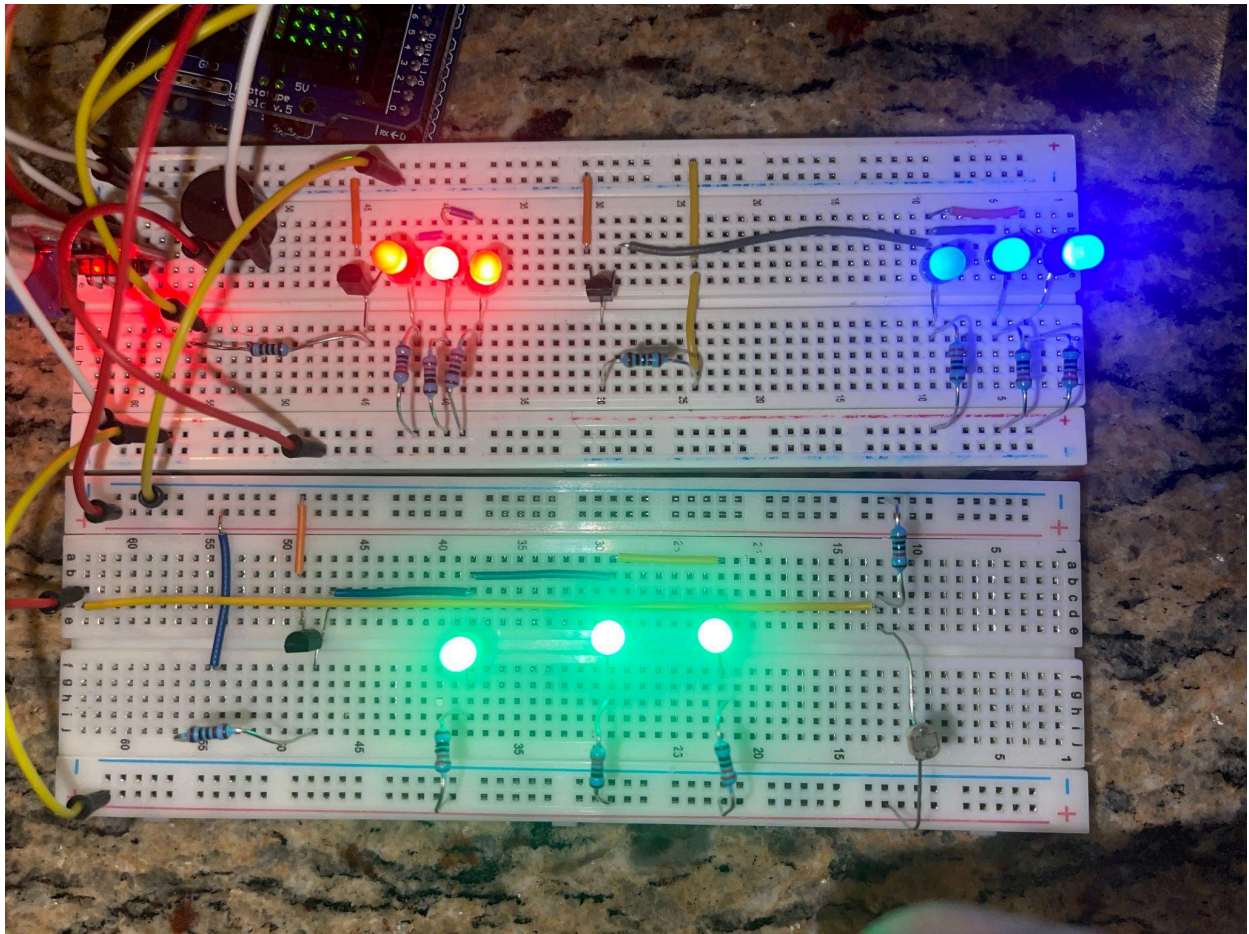


Sound-Triggered Multi-Channel Controller

By: Jhostin Sanchez

Embedded Systems Project

April 2026



Abstract

The sound-triggered multi-channel controller system is an embedded systems project designed to react to sound inputs and serial commands to control multiple output channels. The system is built around an Arduino Uno R3 and uses a sound sensor as the main input method, along with serial communication for additional control.

The system drives three LED channels (red, green, and blue) and a buzzer using transistor-based switching, allowing the microcontroller to safely control multiple outputs. Different input patterns, like single or multiple claps, trigger different combinations of outputs, making the system responsive and structured.

This project was built to help me understand how embedded systems handle real-world inputs and turn them into outputs through hardware and code working together. It focuses on learning how signals are processed, how transistors are used as switches, and how a full system can be organized from input to output in a simple but functional design.

Introduction

The purpose of this project is to explore how embedded systems respond to real-world inputs and convert them into controlled outputs using both hardware and software. The system is based on an Arduino Uno R3 and was designed as a hands-on way to understand how input signals, processing logic, and output devices work together in a complete circuit.

This project focuses mainly on learning how different types of inputs like sound detection through a microphone sensor and commands sent through serial communication can be interpreted by a microcontroller and used to control multiple outputs. In this case, three LED channels and a buzzer are activated in different patterns depending on the input received.

A major part of this build was understanding transistor behavior as electronic switches. Instead of driving all components directly from the microcontroller, transistors were used to safely handle multiple output channels. This helped reinforce how embedded systems are not just about coding, but also about proper hardware design and signal control.

Overall, this project was developed to strengthen practical knowledge in embedded systems, especially in input processing, output control, and system-level thinking where hardware and software need to work together efficiently.

Objectives

- Design a simple embedded system that responds to real-time sound inputs and serial commands using an Arduino Uno R3
- Understand how different input types (sound sensor and serial communication) are processed by a microcontroller
- Implement multi-channel output control using three LED channels (red, green, blue) and a buzzer
- Explore transistor behavior as switching components for controlling external loads safely
- Develop event-based control logic where different input patterns trigger different output combinations
- Strengthen understanding of embedded systems design, including input processing, output control, and hardware-software integration

Components & Materials

Component	Quantity	Function
Arduino Uno R3	1	Acts as the main microcontroller, processing inputs and controlling outputs
Sound Sensor Module	1	Detects sound (claps) and sends input signals to the microcontroller
LEDs (Red, Green, Blue)	9	Provide visual output across three independent color channels
NPN Transistors	3	Act as electronic switches to control each LED channel safely
Resistors	12	Limit current for LEDs and protect transistor/base connections
Buzzer	1	Produces sound output when triggered by the system
Breadboards	2	Provide a platform for building and testing the circuit without soldering
Jumper Wires	27	Create electrical connections between components in the circuit
USB Cable	1	Supplies power and enables serial communication with the Arduino

System Overview

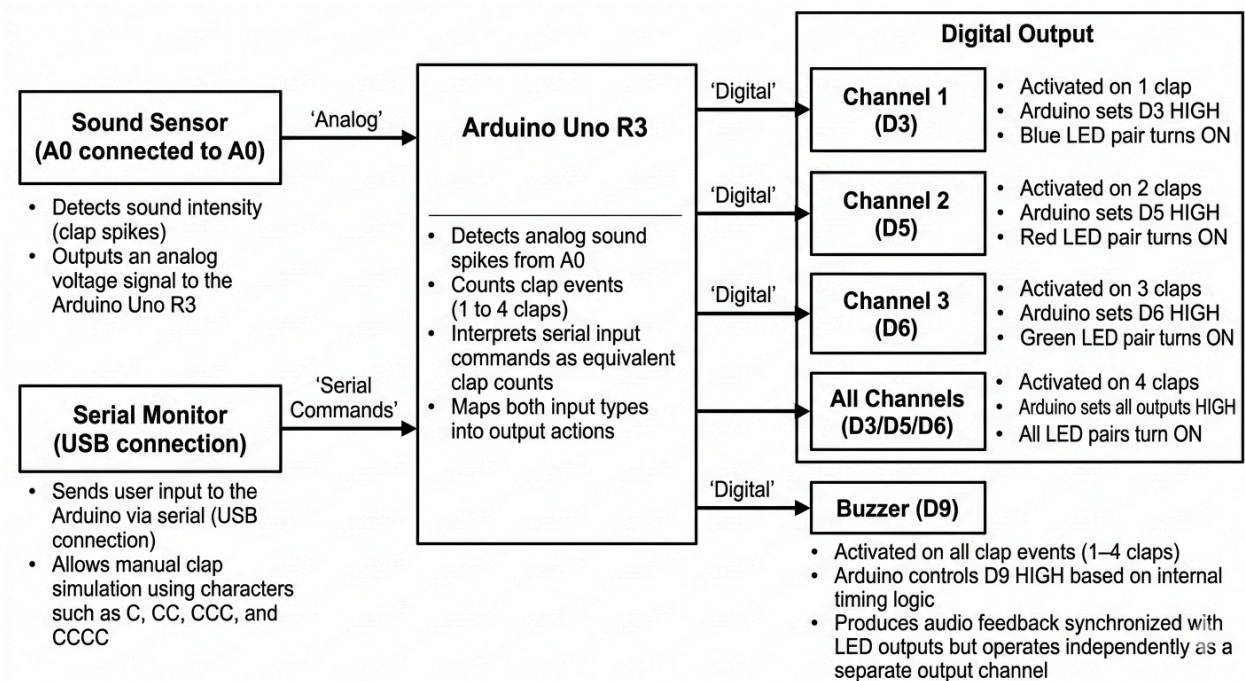
The sound-triggered multi-channel controller system is designed to process input signals and control multiple outputs through a centralized microcontroller. The system is built around an Arduino Uno R3, which receives input from a sound sensor and serial communication, then processes these signals to determine the appropriate output response.

The sound sensor detects acoustic events such as claps and sends a signal to the Arduino. Based on the number or pattern of detected inputs, the microcontroller executes predefined logic to activate specific output channels. In addition to sound input, serial commands can also be used to trigger the system, providing an alternative method of control.

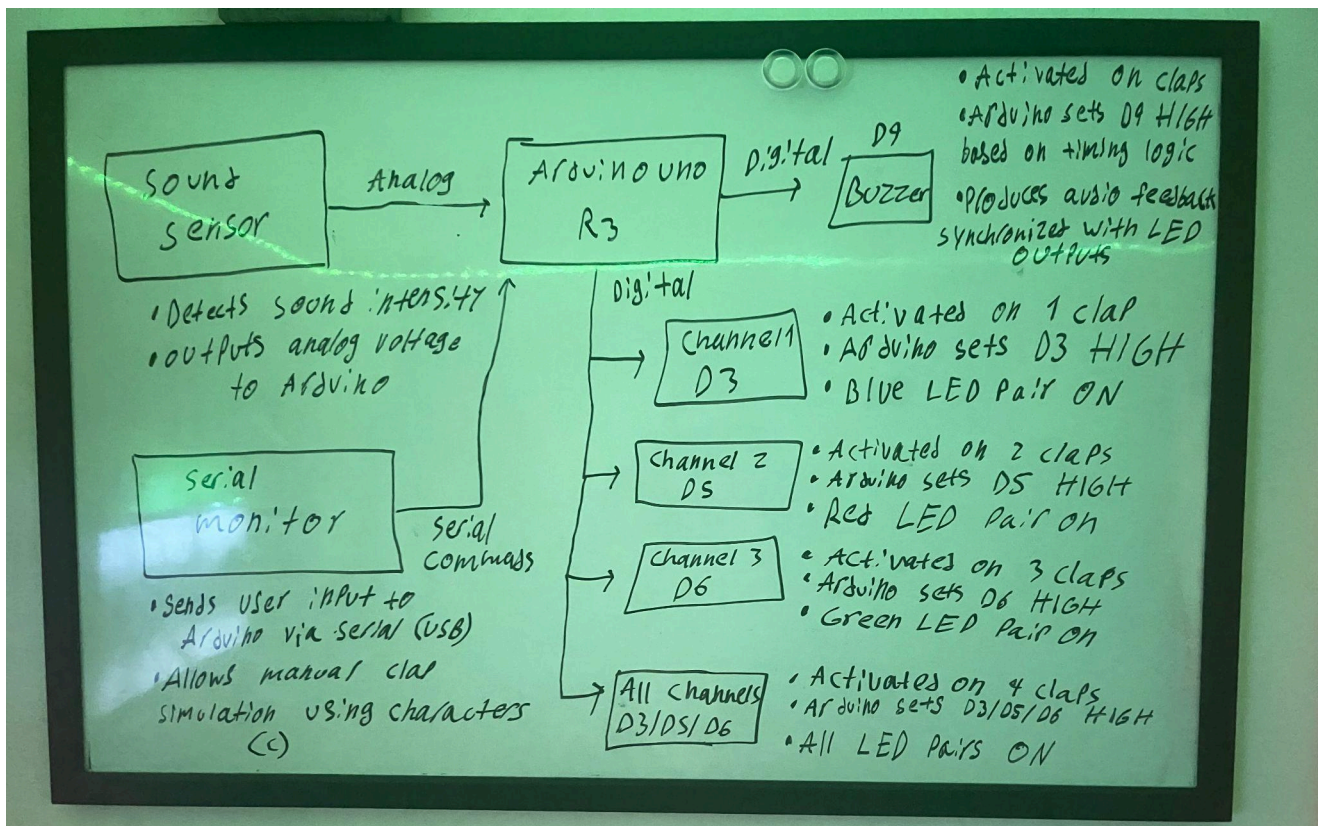
The output stage consists of three independent LED channels (red, green, and blue) and a buzzer. Each LED channel is controlled through a transistor, allowing the Arduino to safely switch higher current loads without directly powering them from its pins. The buzzer is activated alongside certain output states to provide audio feedback.

Overall, the system operates as a multi-input, multi-output embedded controller where input signals are interpreted, processed, and translated into structured output behavior. This demonstrates how hardware and software work together to create a responsive and organized embedded system.

Block Diagram



Physical Block Diagram



Circuit Design

The circuit is designed to interface input, processing, and output stages through the Arduino Uno R3. The sound sensor provides an input signal to the microcontroller, which is used to detect acoustic events such as claps. This signal is processed in software to determine the appropriate output response.

The output stage is built using three NPN transistors, each controlling a separate LED channel (red, green, and blue). The transistors act as switches, allowing the Arduino to control higher current loads without directly powering them from its digital output pins. This protects the microcontroller and ensures stable operation when multiple outputs are active.

Resistors are used throughout the circuit to limit current and protect components. Each LED is connected with a current-limiting resistor to prevent damage, while additional resistors are used at the transistor base to regulate the input signal. The buzzer is connected as an output device and is activated alongside specific LED states to provide audio feedback.

Overall, the circuit is structured to separate input sensing, signal processing, and output control, allowing the system to operate reliably and efficiently.

Circuit Design - Circuit Calculations

This section describes the electrical behavior of the multi-channel LED driver system. Each channel (Red, Green, Blue) operates independently but follows identical circuit topology, differing only in LED forward voltage characteristics.

A. Channel Architecture

Each channel consists of:

- One GPIO control signal (5V logic)
- One 1 k Ω base resistor
- One NPN transistor (low-side switch)
- Three LED branches in parallel
- Each branch contains:
 - One 220 Ω resistor
 - One LED (color-dependent)
- Shared 5V supply rail
- Common ground reference through emitter

B. Operating Principle

When the GPIO signal is HIGH, the transistor enters saturation mode, connecting the LED network to ground. This allows current to flow from the 5V rail through each LED branch independently.

When the GPIO signal is LOW, the transistor enters cutoff mode, stopping all current flow through the LED network.

C. LED Electrical Model

Each LED branch follows a simplified voltage-drop model:

Component	Typical Value
Supply Voltage	5V
Transistor Drop (ON)	$\sim 0.2\text{V}$
Red LED Forward Voltage	$\sim 2.0\text{V}$
Green LED Forward Voltage	$\sim 2.2\text{V}$
Blue LED Forward Voltage	$\sim 3.2\text{V}$
Resistor	220 Ω

D. Branch Current Analysis

Each LED branch current is determined by:

$$I_{LED} = \frac{V_{cc} - V_{led} - V_{ce}}{R}$$

Results per channel:

Channel	LED Voltage	Current/Branch	Branches	I_{Total} (Channel)
Red	2.0V	12.7mA	3	38.1mA
Green	2.2V	11.8mA	3	35.4mA
Blue	3.2V	7.3mA	3	21.9mA

E. Current Summation Rule

At the transistor collector node, currents from each LED branch combine:

$$I_{total} = I_1 + I_2 + I_3$$

This confirms that total channel current is the sum of all parallel LED branch currents.

F. Base Drive Analysis

The transistor base is driven through a 1 k Ω resistor from a 5V GPIO signal.

Base current is given by:

$$I_B = \frac{I_{GPIO} - V_{BE}}{R_B}$$

With:

- $V_{BE} = -0.7v$
- $R_B = 1k\Omega$

Result:

- $I_B = 4.3mA$

To ensure saturation:

$$I_B \geq \frac{I_C}{10}$$

This condition is satisfied for all channels, ensuring proper switching behavior.

G. Key Electrical Observations

- Voltage is constant across each branch (5V supply)
- Current divides across parallel LED paths
- LED forward voltage determines current magnitude
- Transistor operates strictly as a switching device
- Resistors are the primary current-limiting components

H. Design Conclusion

The circuit successfully implements a multi-channel LED driver using transistor switching. Each channel operates independently and maintains safe current levels under standard operating conditions. Differences in LED forward voltage result in varying brightness levels across colors, which is expected in resistor-based LED driving systems.

I. Theoretical Foundation and Circuit Laws

The electrical behavior of each LED channel is governed by fundamental circuit laws, primarily Kirchhoff's Current Law (KCL) and Ohm's Law.

According to KCL, the total current entering a node must equal the total current leaving the node. In this circuit, the transistor collector node serves as a convergence point where currents from multiple parallel LED branches combine.

Mathematically, this is expressed as:

$$I_{total} = I_1 + I_2 + I_3$$

Each branch current is independently determined by the voltage drop across its resistor and LED. Because all branches are connected in parallel, they share the same supply voltage, but the current through each branch varies depending on the LED forward voltage.

Ohm's Law is applied to each branch:

$$I = \frac{V}{R}$$

Where the effective voltage across each resistor is:

$$V_R = V_{CC} - V_{LED} - V_{CE(sat)}$$

This explains why different LED colors produce different currents, even when identical resistors are used.

J. Design Justification and Component Selection

The resistor value of 220 Ω was selected to balance LED brightness and current safety.

Parameter	Value	Engineering Reason
Supply Voltage	5V	Standard Arduino output level
Target Current	~10-15mA	Safe LED operating range
Resistor Value	220 Ω	Limits current without excessive dimming

If a lower resistance (e.g., 100 Ω) were used:

- Current would increase significantly
- LEDs could overheat or degrade over time

If a higher resistance (e.g., 330 Ω) were used:

- Current would decrease
- LEDs would appear dimmer

Thus, 220 Ω represents a practical compromise between performance and safety.

K. Transistor Operating Limits

The NPN transistor (2N2222) is used as a low-side switch and must operate within safe current limits.

- Maximum collector current: ~600 mA (absolute max)
- Typical operating range: well below this value

From calculations:

- Maximum channel current ≈ 38 mA

This is significantly below the transistor limit, confirming safe operation.

Additionally, base current is intentionally chosen to ensure saturation:

$$I_B \geq \frac{I_C}{10}$$

This guarantees that the transistor behaves as a fully ON switch rather than operating in the linear region.

L. Engineering Insight

This circuit demonstrates how a simple parallel network can produce different electrical behaviors based on component characteristics. Even with identical resistors, variations in LED forward voltage directly impact current distribution.

This highlights an important engineering principle:

Component selection directly influences system behavior, even in structurally identical circuits.

Pin Configuration

Sound Sensor

Arduino Uno R3

VCC	5V
GND	GND
AO	A0
DO	UNCONNECTED

Buzzer

Arduino Uno R3

Positive(+)	Digital 9
Negative(-)	GND

LED

Breadboard (Anode)

PN2222 (Cathode)

Blue	5V (Via 220 Ohm Resistor)	Collector (CH1)
Red	5V (Via 220 Ohm Resistor)	Collector (CH2)
Green	5V (Via 220 Ohm Resistor)	Collector (CH3)

PN2222 Transistor

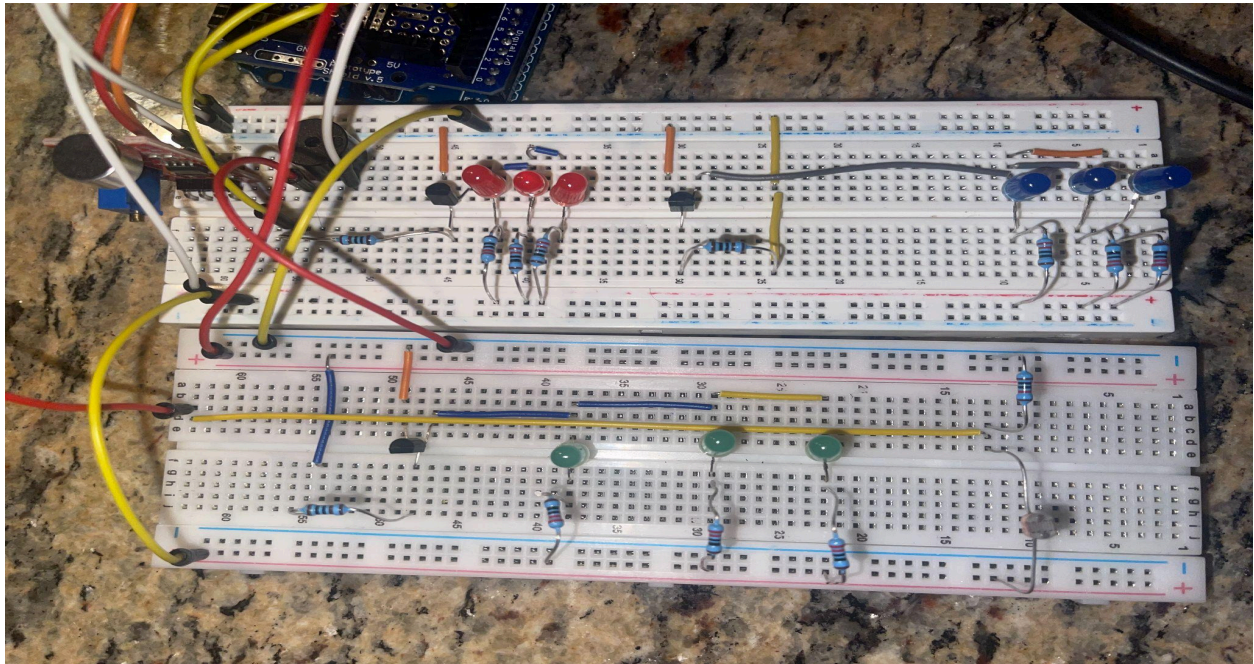
LED (Collector)

Arduino Uno R3 (Base)

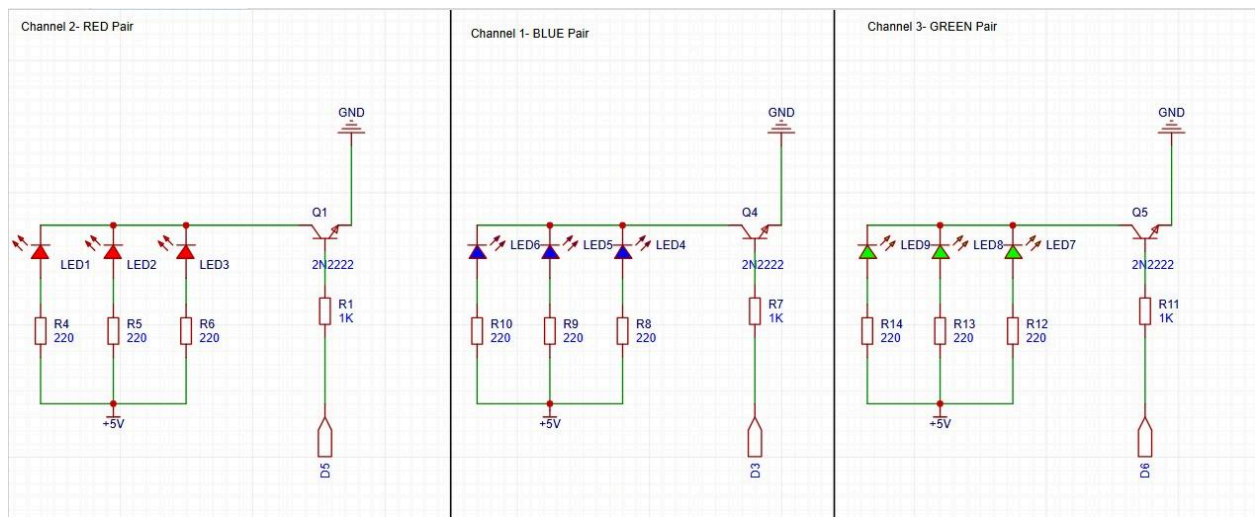
Breadboard (Emitter)

Cathode (Blue Pair)	D3 (Via 1k Ohm Resistor)	GND
Cathode (Red Pair)	D5 (Via 1k Ohm Resistor)	GND
Cathode (Green Pair)	D6 (Via 1k Ohm Resistor)	GND

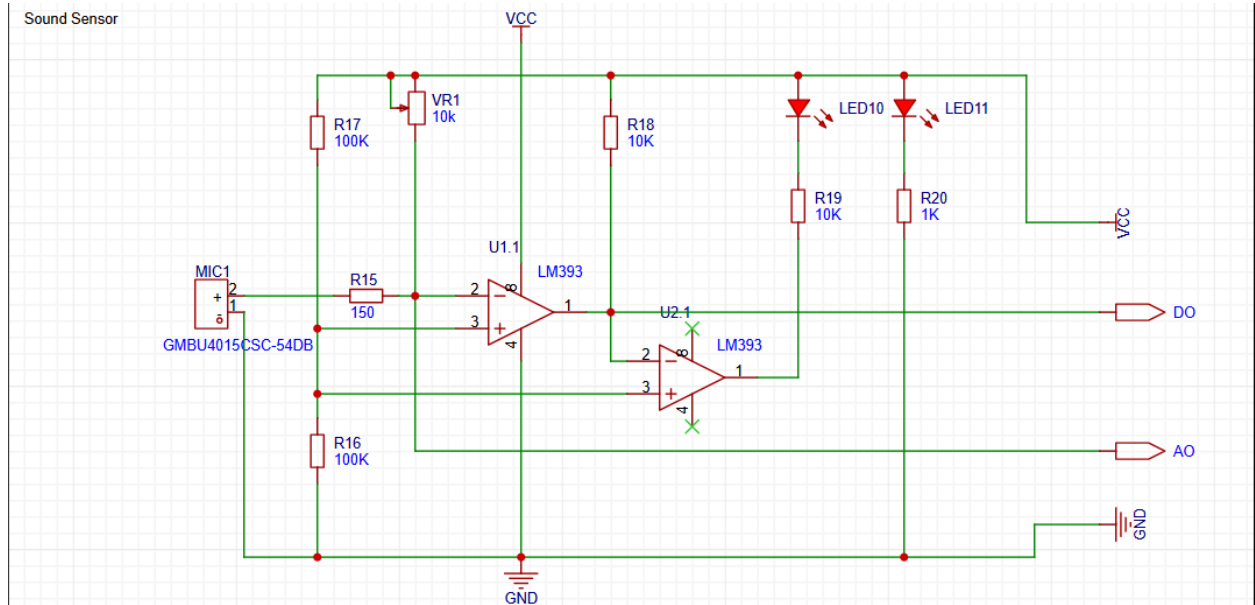
Physical Prototype



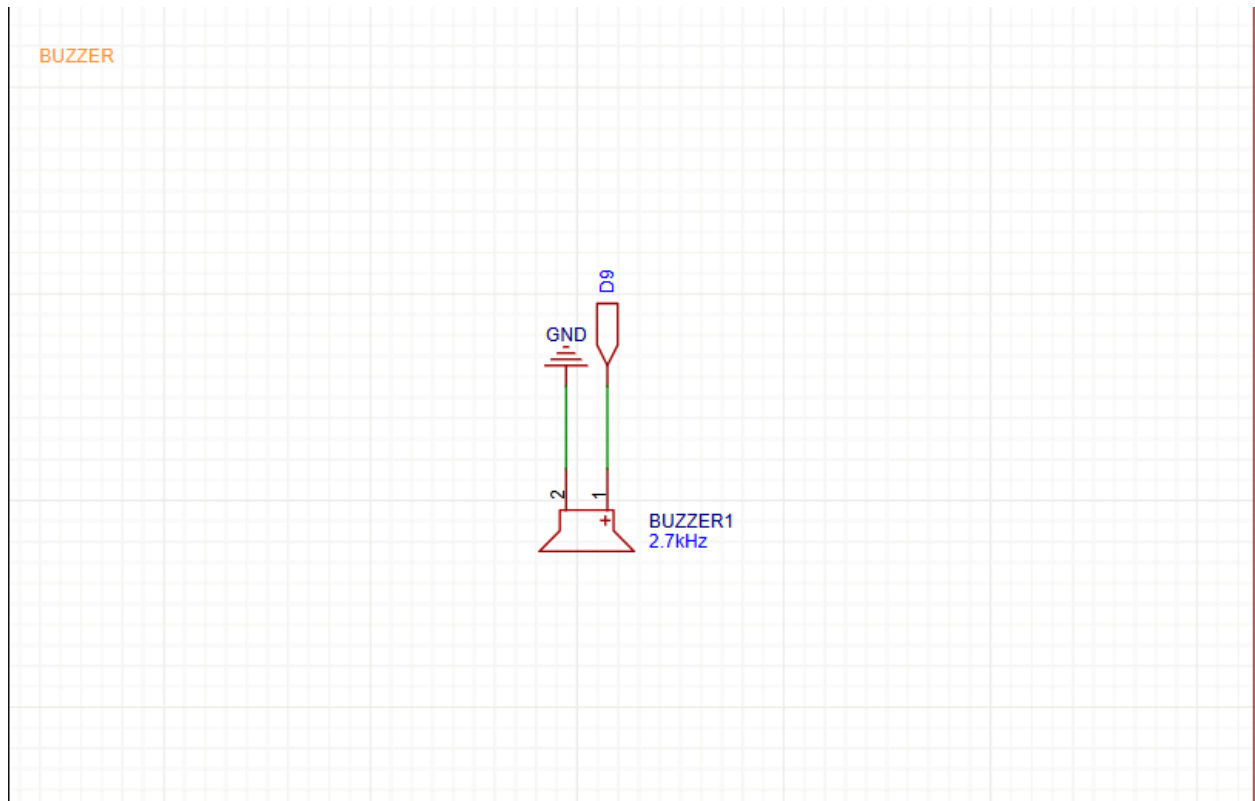
Channel Schematic



Sound Sensor Schematic



Buzzer Schematic



Software Design

The software for this system was developed using the Arduino IDE and written in C/C++. The program is responsible for reading input signals from the sound sensor and serial communication, then translating those inputs into specific output behaviors in real time.

The main logic continuously monitors the sound sensor to detect clap events. Based on the number of detected claps within a short time window, the program determines which LED channels (red, green, or blue) should be activated. Each pattern corresponds to a different output state, allowing the system to respond differently depending on the input.

In addition to sound-based control, the system also accepts serial commands from a connected computer. These commands allow manual triggering of the same output patterns, which was useful for testing and debugging without relying on sound input.

The output logic is structured so that each LED channel is controlled through separate digital pins connected to transistor switches. This ensures that the microcontroller can safely control multiple outputs at the same time while maintaining stable operation. The buzzer is also controlled through software and is activated alongside specific output states for audio feedback.

System Operation

When the system is powered on through the USB connection, the Arduino Uno R3 initializes and begins continuously monitoring both the sound sensor and serial input. At this stage, all output channels remain inactive until a valid input is detected.

When a clap is detected by the sound sensor, the signal is sent to the Arduino, which processes the input based on timing and count. Depending on the number of claps detected within a short window, the system triggers different output combinations. These outputs control the red, green, and blue LED channels, along with the buzzer for audio feedback.

In addition to sound input, the system can also be controlled using serial commands sent from a computer. Commands such as different “C” values are interpreted by the Arduino and mapped to the same output behaviors as the clap inputs. This allows the system to be tested and controlled more consistently without relying only on sound detection.

Each output response activates specific LED channels through transistor switching. The buzzer is activated in certain states to provide confirmation of system response. Once the input event is completed, the system resets and returns to monitoring mode, waiting for the next input.

Testing & Results

Input Type	Test Case	Expected Result	Actual Result	Status
Clap Input	Single Clap	Activate LED Channel 1 + buzzer	Ch1 Activated with buzzer	Pass
Clap Input	Double Clap	Activate LED Channel 2 + buzzer	Ch2 Activated with buzzer	Pass
Clap Input	Triple Clap	Activate LED Channel 3 + buzzer	Ch3 Activated with buzzer	Pass
Clap Input	Four Claps	Activate all LED Channels + buzzer	All channels activated with buzzer	Pass
Serial Command	1C	Activate LED Channel 1	Correct output triggered	Pass
Serial Command	2C	Activate LED Channel 2	Correct output triggered	Pass
Serial Command	3C	Activate LED Channel 3	Correct output triggered	Pass
Serial Command	4C	Activate all LED Channels + Buzzer	All outputs activated correctly	Pass
System Behavior	After Input Event	Return to idle monitoring state	System reset successfully after each input	Pass

The testing phase was carried out to verify that the system responds correctly to both sound-based inputs (claps) and serial commands. The main goal of this stage was to confirm that each input type consistently triggers the correct LED channel behavior and that the system operates reliably under repeated use.

Multiple test cases were performed to evaluate individual clap counts, serial command inputs, and full-system activation. Each test was designed to check whether the Arduino correctly processed the input signal, executed the programmed logic, and activated the corresponding output channels (LEDs and buzzer).

The results confirmed that the system behaves as expected in all scenarios. Each clap input successfully triggered its assigned LED channel, while four claps activated all channels simultaneously along with the buzzer. Serial commands produced identical output behavior, confirming that both input methods are fully functional and consistent. After each test, the system returned to idle monitoring mode, ensuring stable and repeatable operation.

Challenges & Limitations

During the development of this project, several challenges were encountered in both the hardware and software design phases.

One of the first issues came from schematic interpretation. At the beginning, there was confusion between the collector and emitter of the PN2222 transistor due to its physical pin layout versus its schematic representation. This caused initial misunderstanding in how the transistor should be connected in the circuit, but it was later corrected after reviewing proper NPN transistor operation in schematic form.

Another challenge occurred in the software design, specifically in coordinating multiple outputs with different input types. The system needed to respond to both sound sensor inputs and serial commands, while also ensuring that each LED channel and the buzzer responded correctly and independently without timing conflicts.

A major issue was related to sound sensitivity. Initially, the sound sensor was not reliably detecting claps because the input threshold was too high. This was resolved by adjusting the potentiometer on the sound sensor and refining the software sensitivity logic to improve detection accuracy.

Additionally, there was an early design misunderstanding regarding the number of transistors required. It was initially assumed that each LED would require its own transistor, but the design was later optimized by grouping LEDs into three channels (red, green, and blue), each controlled by a single transistor. This simplified the circuit and improved overall organization and efficiency.

Future Improvements

Several improvements could be made to expand the functionality and capability of this embedded system.

One possible upgrade is the integration of infrared (IR) communication to allow wireless remote control of the LED channels. This would replace or complement the current serial input method and provide a more flexible user interface.

Another major improvement would be upgrading the system to an ESP32 microcontroller. This would allow the system to support Wi-Fi or Bluetooth communication, enabling wireless control from a mobile device or network-based input system.

A light-based control feature could also be added using a photoresistor (LDR). This would allow the system to respond to environmental lighting conditions, where LED channels could activate based on surrounding brightness or darkness levels.

Additionally, a temperature-based control system could be implemented using a temperature sensor. In this version, the number of active LED channels would depend on temperature thresholds—for example, low temperature activates one channel, moderate temperature activates two channels, and high temperature activates all three channels. This would transform the system into an environmental monitoring and response device.

Conclusion

This project demonstrates the design and implementation of a multi-channel embedded control system that integrates real-world input processing with structured output behavior. By combining hardware design with software logic, the system is able to reliably interpret sound-based inputs and translate them into controlled electrical responses.

A key focus of this project was understanding how electrical principles apply in practical circuit design. The use of transistor-based switching allowed for safe and scalable control of multiple output channels, while circuit analysis using Ohm's Law and Kirchhoff's Current Law provided a deeper understanding of current flow and voltage distribution within the system.

Throughout development, several design challenges were encountered, including transistor configuration errors and signal sensitivity issues. Resolving these problems required both theoretical understanding and practical debugging, reinforcing the importance of accurate circuit interpretation and iterative testing in engineering design.

This project reflects not only the ability to build a functioning embedded system, but also the ability to analyze, justify, and improve it based on electrical principles. It represents a step toward developing a deeper understanding of how hardware and software interact in real-world engineering systems.

Appendix

The full source code for this project is available in the GitHub repository linked below. It includes all Arduino logic used for processing sound inputs, serial commands, and controlling the LED channels and buzzer output.

@JS2028EE - GitHub